

Data Design

Comp Sci 214, Spring 2025

Don't forget to check in on Youhere!

Heads Up: Active Lecture

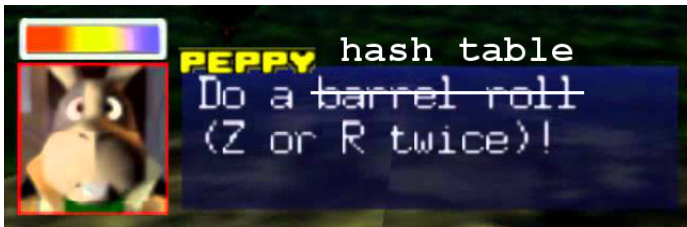
- Today, I want you to do some work during lecture
- When prompted, discuss with the person next to you
- Then share the results with the class
- You'll learn a lot more coming up with your own solutions!
 - ▶ ...compared to passively waiting for your classmates'

Real Programming

Work as a software engineer $\neq$ 214 homeworks

No one's going to ask you to just write a hash table on its own.

(Or even tell you directly to use one!)



Not real programming.

You'll be facing real-world problems, which you'll have to solve.

You'll need to figure out what DSs to use, when, and for what.

Data Design

Before solving a problem, map out the kinds of data you'll need.

- Building a *model*.
- Provides an outline for your program.
- Then implementation is “just” filling out that outline.
 - ▶ “Show me your tables, and I won't usually need your flowcharts; they'll be obvious.” — Fred Brooks

Data Design

Before solving a problem, map out the kinds of data you'll need.

- Building a *model*.
- Provides an outline for your program.
- Then implementation is “just” filling out that outline.
 - ▶ “Show me your tables, and I won't usually need your flowcharts; they'll be obvious.” — Fred Brooks

Naturally an open-ended process!

- Many possible right answers!
- But some may be better than others

Today: we walk through that process on an example problem

Final project: you apply that process to a *much larger* problem

Extended example: Metropolitan areas

The problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

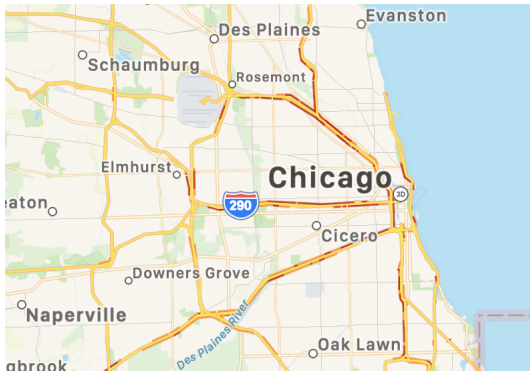
- From Wikipedia: A **metropolitan area** is a region consisting of a densely populated urban core and its less-populated surrounding territories.
- I.e., a group of cities connected to a central one.

The problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

For example:

- **Chicago:** Chicago, Evanston, Cicero, Des Plaines, etc.

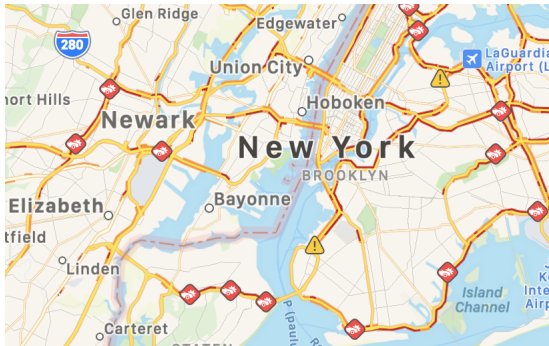


The problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

For example:

- **New York:** New York City, Newark, Union City, etc.

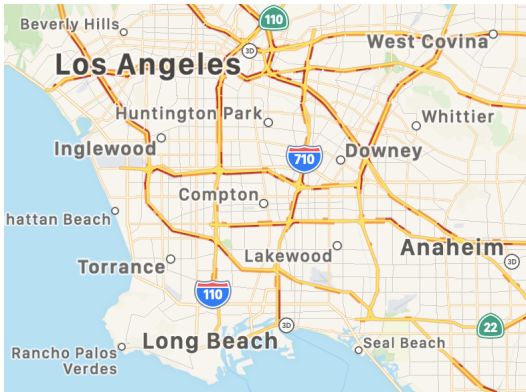


The problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

For example:

- **Los Angeles:** Los Angeles, Anaheim, Long Beach, etc.



The problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

The source data we have available is:

- A collection of information about all US cities (name, population, surface area, etc.)
- Road network data: which cities are connected by roads
- Census data reporting demographic information, median income, educational attainment, etc.

From that data, we'll need to discover which cities are part of which metro areas, compute their total population, etc.

Before You Start

- Make sure you understand the whole problem.
 - ▶ Listen to your customer, in full.
 - ▶ Read the whole specification, in detail.
 - ▶ Etc.
- Then think! Plan a course of action.

Before You Start

- Make sure you understand the whole problem.
 - ▶ Listen to your customer, in full.
 - ▶ Read the whole specification, in detail.
 - ▶ Etc.
- Then think! Plan a course of action.
- Don't jump straight into programming!
 - ▶ **Best case:** you lucked out and ended up with a working solution, but picked up a bad habit along the way.

Before You Start

- Make sure you understand the whole problem.
 - ▶ Listen to your customer, in full.
 - ▶ Read the whole specification, in detail.
 - ▶ Etc.
- Then think! Plan a course of action.
- Don't jump straight into programming!
 - ▶ **Best case:** you lucked out and ended up with a working solution, but picked up a bad habit along the way.
 - ▶ **Likely case:** you write a bunch of code you don't need and throw it out, wasting both time and effort.

Before You Start

- Make sure you understand the whole problem.
 - ▶ Listen to your customer, in full.
 - ▶ Read the whole specification, in detail.
 - ▶ Etc.
- Then think! Plan a course of action.
- Don't jump straight into programming!
 - ▶ **Best case:** you lucked out and ended up with a working solution, but picked up a bad habit along the way.
 - ▶ **Likely case:** you write a bunch of code you don't need and throw it out, wasting both time and effort.
 - ▶ **Worst case:** you write a bunch of code you don't need, can't bring yourself to delete it, and contort the rest of your code to use it even though it's clearly the wrong thing.
- Days of programming can save whole minutes of planning;

Data Design, A Recipe

1. Understand the *problem*, and break it up into sub-goals
2. Identify the data we need to solve the problem
 - Our *entities*
3. Determine how these entities connect with each other
 - Our *relationships*
4. Map each entity and relationship to an *ADT* or a kind of box
5. For each ADT, choose a *data structure* to represent it
6. Start writing your *implementation*, using the above
7. Go back to step 1 when you need data you didn't plan for

Step 1: The Problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

Subgoals

- Determine which cities belong to which metro areas
- Find the n largest metro areas by population

Step 1: The Problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

Subgoals

- Determine which cities belong to which metro areas
- Find the n largest metro areas by population

Let's clarify a few things.

Step 1: The Problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

Subgoals

- Determine which cities belong to which metro areas
- Find the n largest metro areas by population

Let's clarify a few things.

- How do we determine if a city is in metro area X?
 - ▶ The city must be connected by road to city X (center)
 - ▶ And it must be smaller (less population) than city X

Step 1: The Problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

Subgoals

- Determine which cities belong to which metro areas
- Find the n largest metro areas by population

Let's clarify a few things.

- How do we determine if a city is in metro area X?
 - ▶ The city must be connected by road to city X (center)
 - ▶ And it must be smaller (less population) than city X
- Can metro areas overlap?
 - ▶ In reality, no. But for simplicity, we'll allow it here

Step 1: The Problem

We want to group cities into metropolitan areas, then find the n largest metropolitan areas in the US by population.

Subgoals

- Determine which cities belong to which metro areas
- Find the n largest metro areas by population

Let's clarify a few things.

- How do we determine if a city is in metro area X?
 - ▶ The city must be connected by road to city X (center)
 - ▶ And it must be smaller (less population) than city X
- Can metro areas overlap?
 - ▶ In reality, no. But for simplicity, we'll allow it here
- Should a city be included in its own metro area?
 - ▶ Yes!
- Anything else we should clarify before moving to step 2?

Step 2: Our Entities

Take five minutes to discuss with your neighbor what kinds of data you think we'll need to represent to solve our problem.

- Think: “nouns”, “things” in the *domain* (i.e., metro areas)
- We're not thinking about ADTs or data structures yet!

Then report back and let's brainstorm.

Step 2: Our Entities

Take five minutes to discuss with your neighbor what kinds of data you think we'll need to represent to solve our problem.

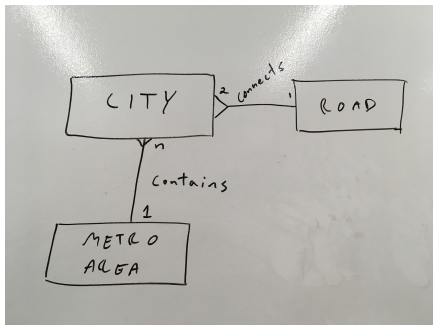
- Think: “nouns”, “things” in the *domain* (i.e., metro areas)
- We're not thinking about ADTs or data structures yet!

Then report back and let's brainstorm.

What I came up with:

1. Cities
2. Connections between cities
3. Metropolitan areas

Step 3: Our Relationships



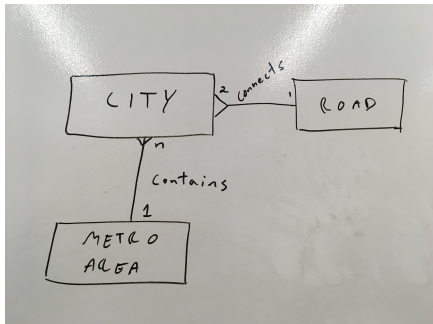
Entity-relationships diagram: maps out

1. the kinds of data we're working with (*entities*)
2. and how they relate to one another (*relationships*)

Each entity box represents a *kind* of data.

We could have multiple *instances* of each! Multiple cities, etc.

Step 3: Our Relationships



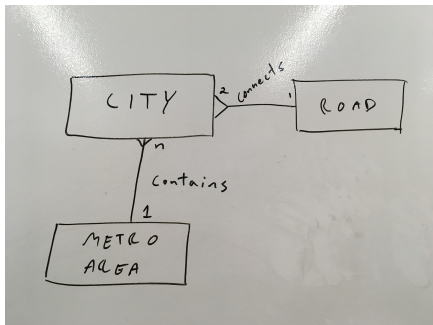
Entity-relationships diagram: maps out

1. the kinds of data we're working with (*entities*)
2. and how they relate to one another (*relationships*)

Each relationship has a *cardinality*.

How many instances of each does it relate: 1-1, 1-n, n-m, etc.

Step 3: Our Relationships

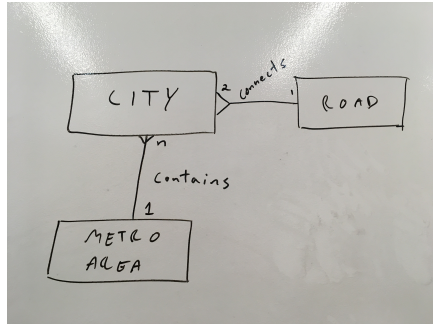


Entity-relationships diagram: maps out

1. the kinds of data we're working with (*entities*)
2. and how they relate to one another (*relationships*)

Each road *connects* two cities. That's a 1-to-2 relationship.
Each metro area *contains* n cities. 1-to- n relationship.

Step 3: Our Relationships

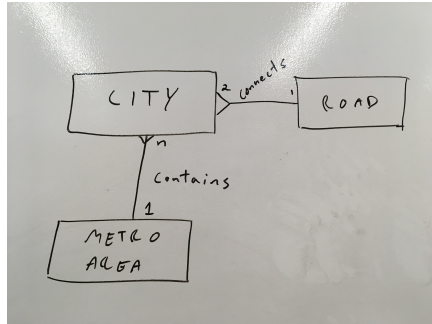


Good first guesses for kinds of data we'll need in our program.

We may discover more as we go along.

Or some may turn out to be unnecessary.

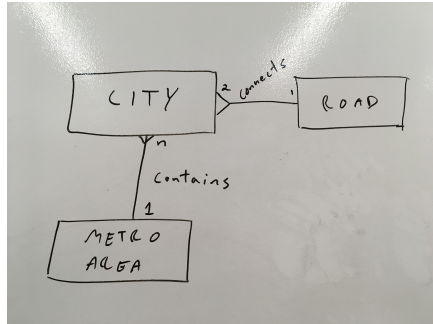
Step 3: Our Relationships



Heads up: you'll have to draw one for the project. Due early!

See notation cheat sheet and reference on Canvas.

Step 3: Our Relationships



Note: some people get very serious about diagrams, exact symbols used, rules for what can go where, etc.

I prefer a more informal approach.

Where did demographic data go?

Wait, we also had some demographic data available!

Just because you have data doesn't mean you should use it!

- **Obvious:** it may not help you solve your problem.
 - ▶ Here, demographic data isn't actually useful.

Where did demographic data go?

Wait, we also had some demographic data available!

Just because you have data doesn't mean you should use it!

- **Obvious:** it may not help you solve your problem.
 - ▶ Here, demographic data isn't actually useful.
- **Subtle:** some data can introduce bias into your program.
 - ▶ Biased data in, biased results out.
 - ▶ E.g., using historical (i.e., biased!) mortgage data to create a system that evaluates mortgage applications.

Where did demographic data go?

Wait, we also had some demographic data available!

Just because you have data doesn't mean you should use it!

- **Obvious:** it may not help you solve your problem.
 - ▶ Here, demographic data isn't actually useful.
- **Subtle:** some data can introduce bias into your program.
 - ▶ Biased data in, biased results out.
 - ▶ E.g., using historical (i.e., biased!) mortgage data to create a system that evaluates mortgage applications.
- **Deeper:** should you even *have* that data in the first place?
 - ▶ Is it really ok for a random web site to know your age, location, religious affiliation, etc. so they can sell ads?
 - ▶ **Remember:** privacy is a human right.
 - ▶ United Nations Universal Declaration of Human Rights, Article 12 (1948)
 - ▶ **See also:** Comp Sci 312: Data Privacy

Step 4: Our ADTs

Next, let's figure out how to express these data.

1. Individual cities
2. Individual roads
3. Connections between cities
4. Cities within a metropolitan area
5. Metropolitan areas

Should these be ADTs, structs, vectors? No concrete DSs yet!

Let's brainstorm!

Step 4: Our ADTs

Next, let's figure out how to express these data.

1. Individual cities → **structs**
2. Individual roads
3. Connections between cities
4. Cities within a metropolitan area
5. Metropolitan areas

Should these be ADTs, structs, vectors? No concrete DSs yet!

Let's brainstorm! That's what I came up with.

1. Structs, with name, population, etc. as fields.

Step 4: Our ADTs

Next, let's figure out how to express these data.

1. Individual cities → **dictionary mapping strings to structs**
2. Individual roads
3. Connections between cities
4. Cities within a metropolitan area
5. Metropolitan areas

Should these be ADTs, structs, vectors? No concrete DSs yet!

Let's brainstorm! That's what I came up with.

1. Structs, with name, population, etc. as fields.
But we may want to **look up** a city's information.
That's a clue we may want a dictionary!
City structs as values, and (say) city names as key.

Step 4: Our ADTs

Next, let's figure out how to express these data.

1. Individual cities → **dictionary mapping strings to structs**
2. Individual roads → **struct**
3. Connections between cities
4. Cities within a metropolitan area
5. Metropolitan areas

Should these be ADTs, structs, vectors? No concrete DSs yet!

Let's brainstorm! That's what I came up with.

2. Only interesting to us because they connect cities.
So we just need to represent that part.
Again, a struct can do, with fields for the two connected cities and (just in case) the length of the road.

Step 4: Our ADTs

Next, let's figure out how to express these data.

1. Individual cities → **dictionary mapping strings to structs**
2. Individual roads → **struct**
3. Connections between cities → **WU graph**
4. Cities within a metropolitan area
5. Metropolitan areas

Should these be ADTs, structs, vectors? No concrete DSs yet!

Let's brainstorm! That's what I came up with.

3. Connections? Sounds like a graph!
We're connecting cities → vertices!
Roads are the connections → edges!

Roads are two-way → undirected!
Roads have lengths → weighted!

Step 4: Our ADTs

Next, let's figure out how to express these data.

1. Individual cities → **dictionary mapping strings to structs**
2. Individual roads → **struct**
3. Connections between cities → **WU graph**
4. Cities within a metropolitan area → **set/sequence**
5. Metropolitan areas

Should these be ADTs, structs, vectors? No concrete DSs yet!

Let's brainstorm! That's what I came up with.

4. Sounds like a set!
But in most languages, sequences are easier to work with (e.g., can iterate over more easily).
Even though we don't care about order, sequence is fine.

Step 4: Our ADTs

Next, let's figure out how to express these data.

1. Individual cities → **dictionary mapping strings to structs**
2. Individual roads → **struct**
3. Connections between cities → **WU graph**
4. Cities within a metropolitan area → **set/sequence**
5. Metropolitan areas → **struct**

Should these be ADTs, structs, vectors? No concrete DSs yet!

Let's brainstorm! That's what I came up with.

5. Clearly need the set of cities.
But other interesting information too:
 - ▶ The name of the metro area (let's use name of central city)
 - ▶ The total population: that's what we're interested in!

Sounds like a struct again.

In General

A few hints for when to reach for various ADTs.

- Collections
 - ▶ Many things, order matters → **sequence**
 - ▶ Many things, order doesn't matter → **set**
- Mapping from names to things → **dictionary**
- Things connected to other things → some **graph**
 - ▶ directed/undirected, weighted/unweighted
- To-do lists
 - ▶ First-in first-out → **queue**
 - ▶ Last-in first-out → **stack**
 - ▶ Most-urgent first-out → **priority queue**

If all you need is a container to combine related aspects of a piece of data, a **struct** will do.

Step 5: Our Data Structures

Finally, let's pick concrete data structures.

1. Dictionary from city names to city structs
2. Road graph
3. Set of cities within a metropolitan area

Let's brainstorm!

Step 5: Our Data Structures

Finally, let's pick concrete data structures.

1. Dictionary from city names to city structs
2. Road graph
3. Set of cities within a metropolitan area

Let's brainstorm! That's what I came up with.

1. **Hash table**

No compelling reason to pick something else.

Step 5: Our Data Structures

Finally, let's pick concrete data structures.

1. Dictionary from city names to city structs
2. Road graph
3. Set of cities within a metropolitan area

Let's brainstorm! That's what I came up with.

1. **Hash table**
No compelling reason to pick something else.
2. **Adjacency list graph**
Road graph should be sparse, right?

Step 5: Our Data Structures

Finally, let's pick concrete data structures.

1. Dictionary from city names to city structs
2. Road graph
3. Set of cities within a metropolitan area

Let's brainstorm! That's what I came up with.

1. **Hash table**
No compelling reason to pick something else.
2. **Adjacency list graph**
Road graph should be sparse, right?
3. **Linked list**
Will probably need to grow it as we figure out the contents.

In General

Many different reasons to pick a data structure over another.
Here are a few:

Efficiency

- Duh. One of our focuses this quarter.
- May require predicting workloads! (risky)
 - ▶ Expecting more insertions than reads? Data structure with $\mathcal{O}(1)$ insertions and $\mathcal{O}(n)$ reads is maybe ok.
 - ▶ Expecting more reads than insertions? Definitely not ok!
 - ▶ Expecting the data structure to have few elements?
Constant factors may be more important than asymptotic complexity.

In General

Many different reasons to pick a data structure over another.
Here are a few:

Fitness for purpose

- E.g., will I need to add vertices to my graphs over time?
- My graph library needs to support that!
- Can't use the one from homework 4, then.

In General

Many different reasons to pick a data structure over another.
Here are a few:

Availability

- Do I have a high quality implementation that I can trust?
 - ▶ Won't crash and bring down my whole system.
 - ▶ Won't introduce security vulnerabilities.
 - ▶ Won't introduce legal liabilities (e.g. licensing, HIPAA, etc.).
 - ▶ Etc.
- Standard library $\ggg$ 3rd-party package $>$ GitHub repository
 $>$ Stack Overflow answer $\ggg$ miscellaneous blog post
 $>$ random forum reply from 2005 $\ggg$ generative AI
 - ▶ (For most languages. YMMV.)
- Depending on context, may win over other factors.

In General

Many different reasons to pick a data structure over another.
Here are a few:

When in doubt, keep it simple.

- If it's sufficient, great! Low effort, working solution.
- Later, *if and only if* it's not good enough... (measure it!)
 - ▶ ...*and* it's actually the problem... (profile it!)
 - ▶ ...*then* replace with something more clever. (ADT power!)
- Otherwise you're making your code more complicated (= more bug-prone) for no real benefit!
- Overengineering is the enemy.
 - ▶ (And it preys on eager young developers.)

Step 6: Our Implementation

Let's do this live.

We'll work with a much smaller dataset than the whole US

- A few dozen or so cities in the tri-state area
- So we have something a bit more digestible

Program / Design Evolution

“Walking on water and developing software from a specification are easy if both are frozen.” — Edward Berard

Most complexity in software development arises from change.

- A program / data design that worked well before, may not work anymore with new requirements.
- Changes may require redesigns.
- Alternatively, changes may get shoehorned into existing designs → technical debt.

Possible Requirement Changes for our Example

- Want to get n metro areas with the *largest number of cities*.
- Want to get n metro areas by *total area*.
- Want the n largest metro areas in a *specific state*.
- Want the n largest metro areas in *each state*.

What changes would be required for each?

- Code changes only?
- Data structure changes (within the same ADT)?
- Data design adjustments?
- Complete redesign?

Beware Exclusionary Data Design

Data design defines what data is allowed within a system.

- That's the whole point!
- **Flip side:** everything else is disallowed.
- Need to be careful about what you rule out!

Beware Exclusionary Data Design

Data design defines what data is allowed within a system.

- That's the whole point!
- **Flip side:** everything else is disallowed.
- Need to be careful about what you rule out!

Example: representing people's names

```
struct person:  
    let first_name: string_only_letters?  
    let last_name:  string_only_letters?  
    ...
```

Boom. Done. Right?

Beware Exclusionary Data Design

Data design defines what data is allowed within a system.

- That's the whole point!
- **Flip side:** everything else is disallowed.
- Need to be careful about what you rule out!

Example: representing people's names

```
struct person:  
    let first_name: string_only_letters?  
    let last_name:  string_only_letters?  
    ...
```

Boom. Done. Right?

- Well, what about middle names? Or people with three first names? Or people without a last name? etc.
- Also, no one would ever use a “-” in their name, right?
- <https://www.kalzumeus.com/2010/06/17/falsehoods-programmers-believe-about-names/>

Project Advice

- **START EARLY**

- ▶ It is **big**, and takes a significant amount of time
- ▶ A lot of that time should be *thinking* and *designing*, not writing code
- ▶ **Warning:** You will **need** a plan. No plan, no help in OHs!

- **Three attempts**, instead of the usual two

- ▶ You'll need to make good use of our feedback
- ▶ The closer your solution is to complete, the better the feedback you'll get!
- ▶ Make good use of each submission, don't waste them!

- **Don't forget the design documents!**

- ▶ First one due on the first deadline
- ▶ Details in the handout

Next time: Single-source
shortest paths